# 2. Manual de datos

Especialización en IA Automation - CoderHouse | Gerónimo Pastori | Octubre 2026

## Modelo de datos

La base Airtable "Centro de Comando" tiene tres tablas. Centro de control y Log de errores están vinculadas con una relación uno-a-muchos: una pieza puede tener varios errores registrados, y cada error pertenece a una pieza.

### Tabla Centro de control (piezas)

| Campo | Tipo | Uso |
|---|---|---|
| Idea Semilla | Texto | Entrada de la persona. Es el único dato que se carga a mano. |
| Estado | Selección única | Generando, En revisión, Aprobado, Publicado, Error. Controla el Router. |
| Borrador | Texto largo | Salida del modelo de IA. |
| Aprobado | Casilla | Marca visual de aprobación. |
| Última ejecución | Fecha de última modificación | Campo que observa el disparador de Make. |
| Historial de revisiones | Texto largo | Registro textual del paso por revisión. |
| Fecha de aprobación | Fecha y hora | Se completa al publicar (módulo 11). |
| Notas de publicación | Texto largo | Observaciones de la persona. |
| Responsable de revisión | Usuario | Quién revisa la pieza. |
| Última modificación | Fecha de última modificación | Auditoría. |
| Log de errores | Vínculo a Log de errores | Errores asociados a la pieza. |
| EsError | Fórmula | IF({Estado}="Error",1,0). Su promedio es la tasa de error. |
| EsPublicado | Fórmula | Vale 1 si el Estado es Publicado. Su suma cuenta las piezas publicadas. |

### Tabla Base RAG (conocimiento)

| Campo | Tipo | Uso |
|---|---|---|
| Tema | Texto | Título del registro (tono de voz, público objetivo, perfil, temas, estilo a evitar, estructura, principio de revisión). |
| Contenido Especifico | Texto | Instrucción o dato atómico de ese tema. |
| Categoría | Texto | Estilo, Audiencia, Autor, Contenido o Formato. |

La base tiene 7 registros cortos. Cada uno contiene un solo concepto para que el contexto enviado al modelo sea preciso y no incluya datos de más.

### Tabla Log de errores

| Campo | Tipo | Uso |
|---|---|---|
| Error | Texto | Descripción corta. |
| Módulo | Texto | Dónde falló (Router, Agente de guión). |
| Mensaje | Texto largo | Mensaje técnico o motivo. |
| Tipo | Selección única | Falla de la IA, Datos faltantes. |
| Resuelto | Casilla | Seguimiento. |

| | | |
|---|---|---|
| Fecha | Fecha | Momento del error. |
| Pieza | Vínculo a Centro de control | Pieza afectada. |

## Esquemas JSON de transferencia entre módulos

Los esquemas salen del blueprint exportado (blueprint.json). Los valores entre llaves son variables de Make que se resuelven en cada ejecución.

### Disparador → Router (salida del módulo 1)

```
{
  "id": "recXXXXXXXXXXXXXX",
  "createdTime": "2026-10-24T18:43:45.000Z",
  "Idea Semilla": "string",
  "Estado": "Generando | En revisión | Aprobado | Publicado | Error",
  "Borrador": "string (texto largo)",
  "Aprobado": true,
  "Última ejecución": "date",
  "Historial de revisiones": "string",
  "Fecha de aprobación": "date",
  "Notas de publicación": "string",
  "Responsable de revisión": { "id": "string", "email": "string", "name": "string" },
  "Última modificación": "date"
}
```

### Base RAG → Aggregator (salida del módulo 2, uno por registro)

```
{ "id": "string", "Tema": "string", "Contenido Especifico": "string", "Categoría": "string" }
```

El Aggregator genera un único texto con la plantilla "Tema: {{2.Tema}}. {{2.Contenido Especifico}}" por cada registro; el resultado queda en {{3.text}}.

### Entrada del nodo de IA (módulo 4, Groq)

```
{
  "model": "openai/gpt-oss-120b",
  "messages": [
    { "role": "system",
      "content": "Actuá como redactor de contenido para LinkedIn. Usá únicamente la
información del bloque CONTEXTO ... Devolvé solo el post, sin comentarios.\n\nCONTEXTO:
({{3.text}})" },
    { "role": "user", "content": "IDEA SEMILLA: ({{1.`Idea Semilla`}})" }
  ],
  "max_tokens": 1024,
  "response_format": "text"
}
```

### Salida del nodo de IA → actualización de la pieza (módulo 6)

```
{
  "id": "{{1.id}}",
  "record": {
    "Estado": "En revisión",
    "Borrador": "{{4.result}}",
    "Historial de revisiones": "Borrador generado por IA, pendiente de aprobación"
  }
}
```

### Registro de error (módulos 12 y 15)

```
{
  "table": "Log de errores",
  "record": {
    "Error": "El agente no realizó el guión",
    "Módulo": "Agente de guión",
    "Mensaje": "{{4.error.message}}",
    "Tipo": "Falla de la IA",
    "Pieza": ["{{1.id}}"]
  }
}
```

En la ruta de datos faltantes (módulo 12) los valores son fijos: Error = "Idea semilla vacía", Módulo = "Router", Tipo = "Datos faltantes".

### Aprobación y cierre (módulos 10 y 11)

```
Gmail:  { "to": ["<destinatario>"], "subject": "Pieza Aprobada: {{1.`Idea Semilla`}}",
          "content": "{{1.Borrador}}", "bodyType": "rawHtml" }
Update: { "id": "{{1.id}}", "record": { "Estado": "Publicado", "Fecha de aprobación":
"{{now}}" } }
```

## Reglas de calidad de datos

- Los filtros del Router comparan texto contra texto (Estado) y usan el operador "existe / no existe" para Idea Semilla y Borrador, así no se confunden vacíos con texto.
- Los campos de selección única se mapean eligiendo la opción del menú y no escribiendo el texto, porque Airtable rechaza opciones que no existen (error 422).
- Los errores se vinculan a la pieza por ID de registro, no por nombre, para que la relación no se rompa si cambia el texto de la idea.